# DOTNET

# PROJECT

# IDEA



- **Web Application:** An ASP.NET Core Web App (MVC) developed in C# that includes at least four modules with full database CRUD operations. application must be able to run using IIS localhost server.
- **Mobile Application:** A mobile .NET replication of the first project with working menus, while basic menus are required, the rubric awards higher marks for implementing up to three CRUD modules on mobile.
- **Deliverables:** A formal project report, presentation slides, a demo video, and a compiled source code folder.

# MY PLAN



# Inter-Faculty Varsity Tracker

1. **Sports & Rules Engine Module**
    - Create: Register new sports disciplines (e.g., Futsal, Badminton, Basketball) and define scoring rules (Goals, Sets, Points) with max period limits.
    - Read: View all configured sports and their rulesets in an administrative table.
    - Update: Modify sport parameters, scoring types, or maximum period numbers.
    - Delete: Remove unused or canceled sports disciplines from the database.
2. **Faculties & Player Rosters Module**
    - Create: Add participating university faculties (e.g., FSKTM, FKEE) and register student athletes into team rosters.
    - Read: Display faculty directories, team rosters, and individual player details.
    - Update: Edit faculty manager contacts, update player jersey numbers, or change assigned player positions.
    - Delete: Remove graduated/withdrawn players or remove a faculty entry from competition.
3. **Venues & Facilities Module**
    - Create: Register UTHM sports facilities (e.g., Main Stadium, Court A) with location descriptions.
    - Read: View the list of all campus venues and monitor real-time availability statuses.
    - Update: Edit venue location details or toggle IsAvailable (true/false) when safety maintenance is needed.
    - Delete: Remove decommissioned or obsolete campus venue records.
4. **Match Scheduling & Live Score Command Center**

- Create: Schedule new match fixtures by selecting Home Faculty, Away Faculty, Sport Type, Venue, Date/Time, and assigning a Scorekeeper.
- Read: View complete match schedules, filter fixtures by status (Scheduled, Live, Completed), and inspect detailed set breakdowns.
- Update: Scorekeepers adjust live scores via background AJAX calls (+1, -1), record set scores (MatchScore), and update match statuses (Scheduled → Live → Completed).
- Delete: Cancel scheduled match fixtures or clear invalid match entries.

**5. User & Role Management Module (RBAC)**

- Create: Register system user accounts and assign roles (Admin, Scorekeeper, Student).
- Read: View total user lists filtered by role and inspect matches assigned to specific Scorekeepers.
- Update: Reassign user roles, update contact emails, or reset account credentials.
- Delete: Revoke system privileges or remove inactive user accounts.

**.NET Mobile Application (Student User App)**

**1. Student Profile Management (Mobile CRUD #1)**

- Create: Register a new student user profile during initial app launch.
- Read: View personal account details, matriculation number, and primary faculty preference.
- Update: Edit display name, contact email, profile picture, or default faculty affiliation.
- Delete: Deactivate or permanently delete the mobile student account.

**2. Personal Match Reminders (Mobile CRUD #2)**

- Create: Tap the bell icon on an upcoming match fixture to add it to a personal alert list.
- Read: Display a personalized view listing all saved match fixtures and upcoming start times.
- Update: Adjust notification lead time settings (e.g., change alert from 15 minutes to 1 hour before match kickoff).
- Delete: Remove saved fixtures from the reminder list or toggle off notification alerts.

**3. Venue Feedback & Safety Tickets (Mobile CRUD #3)**

- Create: Submit a facility issue report (e.g., "Court 2 floodlight damaged") linked to a specific venue.
- Read: Monitor the status of submitted feedback tickets (Pending, In Review, Resolved).
- Update: Edit submitted report details, adjust issue categories, or append follow-up notes.
- Delete: Cancel or withdraw a submitted feedback ticket.

# DB

## 1. Identity & Role-Based Access Control (RBAC)

```
public class User

{

    public int UserId { get; set; }                    // Primary Key

    public string MatricNo { get; set; }               // Unique ID (Student / Official / Admin)

    public string FullName { get; set; }

    public string Email { get; set; }

    public string PasswordHash { get; set; }

    public string Role { get; set; }                   // "Admin", "Scorekeeper", or "Student"


    // Optional link if user belongs to a specific faculty

    public int? FacultyId { get; set; }

    public Faculty Faculty { get; set; }


    // Navigation Properties

    public ICollection<Match> AssignedMatches { get; set; }   // Matches assigned to this Scorekeeper

    public ICollection<MatchReminder> MatchReminders { get; set; } // Student Reminders

    public ICollection<VenueFeedback> VenueFeedbacks { get; set; } // Student Feedback Tickets

}
```

## 2. Tournament Configuration & Roster Entities

```
public class Faculty
{
    public int FacultyId { get; set; }                  // Primary Key
    public string FacultyCode { get; set; }             // e.g., "FSKTM", "FKEE"
    public string FacultyName { get; set; }             // Full Title
    public string ManagerName { get; set; }

    public ICollection<Player> Players { get; set; }
    public ICollection<User> FacultyUsers { get; set; }
}

public class Sport
{
    public int SportId { get; set; }                    // Primary Key
    public string SportName { get; set; }               // e.g., "Futsal", "Badminton", "Basketball"
    public string ScoringType { get; set; }             // "Goals", "Sets", "Points"
    public int MaxPeriods { get; set; }                 // e.g., 2 (Halves), 3 (Sets), 4 (Quarters)

    public ICollection<Match> Matches { get; set; }
}

public class Venue
```

```
{
    public int VenueId { get; set; }                    // Primary Key
    public string VenueName { get; set; }               // e.g., "Main
Stadium", "Court A"
    public string Location { get; set; }
    public bool IsAvailable { get; set; }

    public ICollection<Match> Matches { get; set; }
    public ICollection<VenueFeedback> VenueFeedbacks { get; set; }
}

public class Player
{
    public int PlayerId { get; set; }                   // Primary Key
    public int FacultyId { get; set; }                  // Foreign Key -> Faculty
    public string FullName { get; set; }
    public string MatricNo { get; set; }
    public string PositionRole { get; set; }            // e.g., "Striker",
"Point Guard"

    public Faculty Faculty { get; set; }
}
```

## 3. Match Engine & Scoring Tables

```
public class Match
{
    public int MatchId { get; set; }                    // Primary Key
    public int SportId { get; set; }                    // Foreign Key -> Sport
    public int HomeFacultyId { get; set; }              // Foreign Key -> Faculty (Home)
    public int AwayFacultyId { get; set; }              // Foreign Key -> Faculty (Away)
    public int VenueId { get; set; }                    // Foreign Key -> Venue

    // Assigned Match Official (Scorekeeper Role)
    public int? ScorekeeperId { get; set; }             // Foreign Key -> User (Role = "Scorekeeper")
    public User Scorekeeper { get; set; }

    public DateTime MatchDate { get; set; }
    public string Status { get; set; }                  // "Scheduled", "Live", "Completed", "Cancelled"
    public int HomeTotalScore { get; set; }
    public int AwayTotalScore { get; set; }

    // Foreign Key Relationships
    public Sport Sport { get; set; }
    public Faculty HomeFaculty { get; set; }
    public Faculty AwayFaculty { get; set; }
    public Venue Venue { get; set; }
```

```
    // Breakdown scores per set/half
    public ICollection<MatchScore> MatchScores { get; set; }
    public ICollection<MatchReminder> MatchReminders { get; set; }
}

public class MatchScore
{
    public int MatchScoreId { get; set; }            // Primary Key
    public int MatchId { get; set; }                 // Foreign Key -> Match
    public int PeriodNumber { get; set; }            // 1 (Set 1 / 1st Half),
2, 3, etc.
    public int HomePoints { get; set; }
    public int AwayPoints { get; set; }

    public Match Match { get; set; }
}
```

## 4. Mobile Student Interactivity Tables

```
public class MatchReminder
{
    public int ReminderId { get; set; }              // Primary Key
    public int UserId { get; set; }                  // Foreign Key -> User (Role = "Student")
    public int MatchId { get; set; }                 // Foreign Key -> Match
    public int LeadTimeMinutes { get; set; }         // e.g., 15, 30, 60 minutes before match
    public DateTime CreatedAt { get; set; }

    public User User { get; set; }
    public Match Match { get; set; }
}

public class VenueFeedback
{
    public int FeedbackId { get; set; }              // Primary Key
    public int UserId { get; set; }                  // Foreign Key -> User (Role = "Student")
    public int VenueId { get; set; }                 // Foreign Key -> Venue
    public string Subject { get; set; }              // e.g., "Lighting Defect"
    public string Details { get; set; }
    public string Status { get; set; }               // "Pending", "In Review", "Resolved"
    public DateTime SubmittedAt { get; set; }
```

```
    public User User { get; set; }
    public Venue Venue { get; set; }
}
```

*FLOW*

**1. System Setup & Data Provisioning (Web App — Admin)**

1. Configure Sports Rules: Admin creates sport entries (Futsal, Badminton, Basketball) defining scoring mechanisms (Goals, Sets, Points) and maximum period limits.
2. Populate Tournament Roster: Admin registers participating Faculties (e.g., FSKTM, FKEE) and imports student Players into their respective rosters.
3. Register Facilities & Accounts: Admin inputs UTHM Venues and creates User accounts, assigning roles (Admin, Scorekeeper, Student).

**2. Match Scheduling & Assignment (Web App — Admin)**

1. Create Fixture: Admin schedules a new match by selecting Home Faculty, Away Faculty, Sport Type, Venue, Date, and Time.
2. Delegate Official: Admin assigns a specific user with the Scorekeeper role to manage that match fixture.
3. Publish Schedule: The match status transitions to Scheduled, making it instantly viewable across both the Web Admin system and the Mobile App.

**3. Student Pre-Match Interaction (Mobile App — Student)**

1. User Profile Setup (Mobile CRUD #1): Student logs in, registers their profile, and selects their primary faculty preference.
2. Set Fixture Alerts (Mobile CRUD #2): Student browses the match schedule and taps to set a Match Reminder (e.g., 15 or 30 minutes before kickoff).
3. Facility Hazard Reporting (Mobile CRUD #3): If a student inspects a court prior to a game and notices damage (e.g., net torn, floodlights down), they submit a Venue Feedback ticket.

**4. Live Game Execution & Scoring (Web App — Scorekeeper)**

1. Access Assigned Dashboard: Scorekeeper logs into the ASP.NET Core MVC web app and opens their My Assigned Matches command center.

2. Initiate Match: Scorekeeper toggles match status from Scheduled to Live.
3. Record Live Events: Scorekeeper uses inline AJAX buttons (+1 Goal, Set 1 Points, -1) to update scores in real-time. EF Core updates the central database without refreshing the browser tab.

**5. Final Synchronization & Operational Maintenance (System-Wide)**

1. Match Completion: Scorekeeper clicks End Match, locking the final scores and updating the overall Inter-Faculty Standings.
2. Push Mobile Updates: Mobile students receive their scheduled reminders and pull updated real-time match results on their read-only schedules.
3. Facility Maintenance Action: Web Admins review submitted Venue Feedback tickets from students. If a safety issue is verified, the Admin toggles IsAvailable = false on that Venue record to prevent future match scheduling until repairs are completed.

UTHM
Universiti Tun Hussein Onn Malaysia

# INTER-FACULTY VARSITY TRACKER: END-TO-END SYSTEM FLOW

BIE33103 DOTNET PROGRAMMING - SEM 1 2026/2027

## WEB APP WORKFLOW (ASP.NET Core MVC)

### 1. ADMIN SETUP (CRUD)

Sports

Faculties

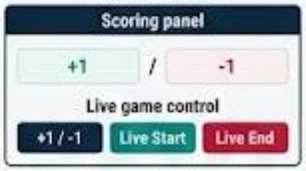
Venues




### 2. MATCH SCHEDULING & ASSIGNING




### 3. SCOREKEEPER LIVE DASHBOARD




Scoring panel UI unttonn
Live game control buttons
will be updated via AJAX.

Read/Write CRUD

AJAX Live Feed

## CENTRAL DATABASE (SQL Server via EF Core)

**User (RBAC Roles)**

| PK | UserID (PK) |
|---|---|
| | Username |
| | PasswordHash |
| | Role |

**Player**

| PK | PlayerID (PK) |
|---|---|
| | MatricNo |
| | FullName |
| | FacultyID |

**MatchScore**

| PK | ScoreID (PK) |
|---|---|
| | MatchID |
| | HomeScore |
| | AwayScore |
| | LogEvents |

**Faculty**

| PK | FacultyID (PK) |
|---|---|
| | FacultyCode |
| | FacultyName |

**Venue**

| PK | VenueID (PK) |
|---|---|
| | VenueName |
| | Location |
| | IsAvailable |

**MatchReminder**

| PK | ReminderID (PK) |
|---|---|
| | MatchID |
| | StudentUserID |
| | NotificationTime |

**Sport**

| PK | SportID (PK) |
|---|---|
| | SportName |
| | Category |

**Match**

| PK | MatchID (PK) |
|---|---|
| | SportID |
| | HomeFacultyID |
| | AwayFacultyID |
| | VenueID |
| | MatchDate |
| | Status |

**VenueFeedback**

| PK | FeedbackID (PK) |
|---|---|
| | VenueID |
| | StudentUserID |
| | FeedbackText |
| | Rating |

Read Only Sync / Subscription Alerts

## MOBILE APP WORKFLOW (.NET Mobile)

### 1. PROFILE MANAGEMENT (CRUD)



### 2. MATCH REMINDERS (CRUD)




### 3. VENUE FEEDBACK TICKETS (CRUD)




### LIVE SCORE VIEWER




*Mobile client utilizes read-only varsity sync with 3 dedicated CRUD modules for profiles, reminders, and feedback. Clean tech-blueprint layout, modern visual nodes, flat vector illustrations.